KNOWLEDGE ENGINEERING AND ONTOLOGIES COURSE PROJECT REPORT

Name & email: Ayşegül YILDIZ & aysegulyildizayse@gmail.com

Name & email: Kerem ÖZAKÇA & keremmozakca@gmail.com

Name & email: Helin HARMAN & helinharman2135@gmail.com

Executive Summary

This project, titled "Ontology Wars: Epic Chronicles," presents an innovative integration of Semantic Web technologies and Large Language Models (LLMs) to create an autonomous, strategy-based simulation. The primary objective is to demonstrate how ontologies can drive complex decision-making processes in dynamic environments. We engineered an OWL-based ontology (T-Box) utilizing METHONTOLOGY principles to define kingdoms, game states (e.g., Safe, Expansion Phase, Under Attack), resources, and diplomatic relations

To overcome the computational limitations and data-type incompatibilities of traditional Description Logic (DL) reasoners (such as HermiT or Pellet) when handling highly dynamic mathematical thresholds, we adopted a robust rdflib-based Python architecture. During the simulation, the engine dynamically generates Knowledge Graphs (A-Box instances) for each turn, which are strictly validated using SHACL. Game logic and AI decisions are purely driven by SPARQL UPDATE queries acting as the core reasoning engine (functionally replacing SWRL rules). Finally, the structured telemetry data from the reasoning engine is fed into the Gemini 2.5 Flash LLM via a Streamlit web interface. This integration bridges formal deterministic logic and generative AI, acting as a "Dungeon Master" to translate semantic state changes into epic natural language narratives.

Description of the project

Project Objective and Problem Definition

The primary objective of this project is to develop a transparent, rule-driven, and formally verifiable decision-making simulation environment named "Ontology Wars: Epic Chronicles," mitigating the black-box limitations of traditional behavioral AI systems. The simulation models a multi-agent geopolitical environment where NPC kingdoms and player entities gather resources, build armies, form alliances, and engage in warfare. In the domain of autonomous systems, decision trees and heuristic algorithms often suffer from rigid hardcoding, poor scalability, and a lack of explicit explainability. This project addresses this challenge by implementing a semantic web-driven core where the state of the world, diplomatic relations, and resource balances are represented via formal logic. By designing a system that relies on explicit ontologies rather than implicit probabilistic models, we ensure that every strategic action taken by an entity can be traced back to mathematical logic and structured assertions, establishing a foundation for verifiable and explainable AI systems.

Theoretical Foundations and Methodologies

The methodological framework of the project is grounded in Ontology Engineering principles and Semantic Web standards established by the W3C. The architecture strictly separates conceptual schema modeling (TBox) from turn-based factual data execution (ABox).

Initially, the project aimed to rely on standard Semantic Web Rule Language (SWRL) rules processed by traditional DL reasoners (e.g., HermiT, Pellet). However, representing highly dynamic game physics—such as continuous resource consumption, famine crises, and threat level fluctuations—exposed the limitations of traditional reasoners in handling frequent state mutations and complex mathematical built-in functions. Consequently, the architecture was completely restructured to utilize a SPARQL-driven reasoning mechanism. The core workflow is now designed as a continuous, dynamic Knowledge Graph construction and reasoning cycle. For each simulation turn, raw telemetry logs are captured, preprocessed, and mapped directly into subject-predicate-object RDF triples. Instead of utilizing external programming logic to dictate entity behavior, the system relies entirely on Semantic Web paradigms implemented via declarative query structures.

Key Components and Technical Architecture

The system architecture comprises six deeply integrated operational components:

- **Ontology Development (TBox Specification):** Modeling the hierarchical structural landscape of the strategy domain using OWL to define foundational classes such as Entity, Player, NPC, GameState, and Resource, alongside complex properties like isAlliedWith and isVassalOf.
- **Data Acquisition and Preprocessing:** Capturing dynamic simulation variables in a structured JSON telemetry format, tracking critical metrics including health points, gold, food, troop counts, and threat levels across consecutive historical ticks.
- **Knowledge Graph Construction:** Utilizing the rdflib Python framework to dynamically load the TBox schema, instantiate ABox individuals from the current turn's telemetry, and bind formal XML schema data types (XSD) to produce a unified, consistent active graph model.
- **Validation Layer:** Integrating SHACL (Shapes Constraint Language) constraints via pyshacl to dynamically execute structural data quality checks (e.g., verifying that entity health remains non-negative and ensuring each kingdom is bound to exactly one logical game state) prior to any reasoning stage.
- **Semantic Reasoning (SPARQL Update Rules):** To overcome the aforementioned DL reasoner limitations, the system executes complex, multi-layered SPARQL UPDATE queries that function as the logical equivalent of SWRL rules. These queries analyze resource thresholds and diplomatic indicators to infer recommended actions, handling scenarios such as defensive fortification during siege states, resource reallocation under famine, or opportunistic alliance breakages.
- **LLM Integration and Narrative Generation:** Connecting the generated graph outputs with a Large Language Model (Gemini 2.5 Flash) inside a centralized Streamlit application. The structured JSON graph states, enriched with SPARQL-derived decisions, are transformed into prompt contexts that guide the LLM to synthesize consistent, high-fidelity epic fantasy prose in both English and Turkish, suppressing hallucination by grounding the narrative strictly within verified ontological facts.

Expected Outputs, Tools, and Target Users

The project delivers a fully reproducible open-source artifact ecosystem consisting of a formalized OWL ontology file, a repository of turn-by-turn serialized Turtle (.ttl) files, a robust Python-based semantic simulation engine, and an interactive Streamlit web dashboard. The core technologies driving the pipeline include Protégé for initial

ontological engineering, RDFlib for graph parsing and programmatic triple manipulation, PySHACL for declarative constraint enforcement, and the Google Generative AI API for contextual language generation. The intended target users include knowledge engineers investigating hybrid AI systems, simulation developers requiring auditable agent behaviors, and computational narrative researchers focusing on data-grounded procedural storytelling.

Competency Questions

To validate the structural adequacy and functional boundaries of the engineered ontology, the system is designed to programmatically resolve the following competency questions via formal querying:

1. Which specific structural GameState subclass (e.g., Safe, ExpansionPhase, UnderAttack) is a given kingdom logically classified into based on its current operational threat level?
2. Which active simulation entities possess an extreme accumulation of economic resources (e.g., Gold >= 40) while simultaneously suffering from a severe deficit in survival assets (e.g., Food <= 20)?
3. What are the explicit identities of all current diplomatic allies or protected vassals bound to a specified kingdom during the active evaluation turn?
4. What is the precisely recommended tactical action inferred by the semantic rules for a kingdom characterized as being "Under Attack" whose military personnel count has fallen below the critical mobilization threshold of 20 troops?
5. Which specific entity individuals or resource nodes are currently triggering data quality violations inside the graph according to the enforced SHACL schema shapes?
6. Which kingdoms exhibit an aggressive expansionist behavior pattern, defined as maintaining a high strategic threat level coupled with a sufficient military troop size?
7. Has an entity initiated an alliance proposal, and if so, does the receiving target entity possess the requisite military power or economic stability to formally accept the pact under the active ontology rules?
8. Which subordinate vassal kingdoms currently possess sufficient monetary funds to allow their structural Overlord entity to exploit their resources for national army recruitment?

Ontology Design

This section presents the conceptual modeling of the "Ontology Wars" domain. The ontology was meticulously designed to formalize the complex interactions, resources, and geopolitical states of autonomous entities within a strategic simulation.

TBox vs. ABox Distinction

A strict distinction is maintained between the conceptual schema (TBox) and the instance-level data (ABox).

- TBox (Terminology Box): Represents the static, immutable structural blueprint of the game world. It includes the definitions of all classes (e.g., Entity, GameState, Resource), object properties, data properties, and the hierarchical relationships between them.
- ABox (Assertion Box): Represents the dynamic, temporal state of the simulation. Because this is a turn-based engine, the ABox is highly volatile. At

the start of each turn, the Python engine explicitly generates new ABox instances (e.g., Kingdom_Avalon, State_Kingdom_Avalon, Troop_Kingdom_Avalon) bound to the TBox schema to represent the exact factual state of that specific tick.

Main Classes

The TBox defines several core classes to categorize the domain:

- **game:Entity:** The root class representing sovereign actors in the simulation. It is partitioned into two disjoint subclasses: game:Player and game:NPC.
- **game:GameState**: Represents the geopolitical and tactical condition of an entity. It includes distinct subclasses based on threat levels: game:Safe (threat <= 3), game:ExpansionPhase (threat 4-7), and game:UnderAttack (threat >= 8).
- **game:Resource:** Represents the tangible assets an entity possesses. It is divided into specific resource subclasses: game:Gold, game:Food, and game:Troop.
- **game:Action:** Represents the strategic decisions available to entities (e.g., Action_Attack, Action_Defend_Castle, Action_OfferAlliance).

Object Properties (Relationships)

Object properties form the semantic network connecting instances:

- **Structural Properties:** game:hasState links an Entity to its GameState. game:hasResource links an Entity to its Resource objects.
- **Diplomatic Properties:** game:isAlliedWith establishes a symmetric alliance between two entities, while game:isVassalOf establishes a directional, hierarchical relationship indicating submission.
- **Interaction Properties:** Properties like game:offersAllianceTo, game:requestsAidFrom, and game:offersVassalageTo represent transient diplomatic actions.
- **Decision Property:** game:recommendedAction links an Entity to the optimal Action inferred by the reasoning engine.

Data Properties

Data properties bind specific literal values (strictly typed as xsd:integer) to instances:

- game:healthPoints: Binds the core survival metric to an Entity.
- game:threatLevel: Binds an integer value to a GameState, determining its subclass classification.
- game:resourceAmount: Binds the quantitative value to a specific Resource instance.

Modeling Decisions and Justifications

Several key ontological design trade-offs were made to ensure the robustness and reasoning capabilities of the system:

1. Classes vs. Instances (Reification of Resources): Instead of simply assigning data properties like hasGoldAmount directly to an Entity, we chose to model resources as distinct classes (Gold, Food, Troop) linked via the hasResource object property. Justification: This reification allows resources to be treated as independent objects that can possess their own properties (like resourceAmount). It vastly simplifies complex SPARQL reasoning rules, enabling the system to query specifically for "Entities that possess a Food

resource object where the amount is less than 20," which is much more scalable if new resource types are added later.

2. Roles vs. Types (Handling Game States): We modeled the tactical state of a kingdom (Safe, UnderAttack) as a separate GameState class linked via hasState, rather than creating subclasses of Entity (e.g., SafeEntity, AttackedEntity). Justification: In ontology engineering, a "Type" is rigid and permanent, whereas a "Role" is anti-rigid and temporary. A kingdom's state fluctuates every turn. If we used subclasses, entities would require constant structural re-classification, which computationally burdens the reasoner. Modeling state as a linked role preserves the structural integrity of the Entity while allowing its tactical condition to change dynamically.
3. Part-Whole Relationships: The hasResource and hasState properties function as mereological (part-whole) relationships. A specific resource instance (e.g., Gold_Kingdom_Avalon) is structurally dependent on and exclusively belongs to its parent entity. This encapsulation ensures that when the reasoning engine calculates economic crises or evaluates vassal taxation, it operates strictly within the bounded scope of that entity's sub-graph, preventing cross-contamination of logic between competing kingdoms.

Data Acquisition

Data Source and Format

Unlike traditional knowledge engineering projects that rely on static web scraping or external APIs, the data for "Ontology Wars: Epic Chronicles" is dynamically and synthetically generated. We developed a custom Python-based simulation engine (engine_v2.py) that acts as an automated, real-time data provider. As the simulation calculates turn-based geopolitical events, resource gathering, and battles, it serializes the active state of the world into a highly structured JSON format.

This data is stored in a continuous log file (telemetry_log.json). Each JSON object in the array represents a discrete chronological "turn" containing the exact state of all active entities. The data fields include integer-based resource metrics (health points, gold, food, troops, threat levels) and string-based relationship arrays (allies, vassalage targets, and recommended actions). Crucially, the log also systematically records the sparql_decisions—the explicit tactical actions logically inferred by the semantic reasoner during that turn—serving as the direct foundational context for the downstream LLM narrative generation.

Preprocessing and Transformation to RDF

Because the data is generated internally rather than scraped from the messy open web, the preprocessing pipeline is directly integrated into the engine's execution loop prior to Knowledge Graph construction. The steps applied are as follows:

- Data Cleaning and Consistency: The engine initializes all kingdoms using a strict structural template (via the create_initial_world function). To prevent null-pointer errors or missing values during RDF mapping, empty fields are handled programmatically at inception (e.g., assigning empty strings "" to vassalOf and empty arrays [] to allies).
- Handling Duplicates: Diplomatic actions and list modifications are safeguarded by conditional checks. For example, before an alliance is formalized, the system checks if the target_id is already present in the entity's allies array, ensuring no duplicate relational data is passed to the graph.
- Transformation for RDF Representation: The transformation from flat JSON to a semantic structure is executed by the build_knowledge_graph function

utilizing the rdflib library. The JSON key-value pairs are iteratively mapped to semantic triples (subject-predicate-object). String identifiers (e.g., "Kingdom_Avalon") are concatenated with the custom game namespace to form unique URIRefs. Crucially, numerical values from the JSON are explicitly cast to Python integers and typed as xsd:integer literals in the graph. This rigid datatype transformation is essential for the subsequent SPARQL reasoning rules, which rely heavily on mathematical filtering (e.g., FILTER(?amt >= 40)).

Data Quality and Limitations

The primary strength of the generated dataset is its exceptional quality and structural uniformity. Because the data production is strictly controlled by the Python engine, it is free from the noise, formatting inconsistencies, and missing values typically encountered in scraped, real-world datasets.

However, there are inherent limitations. The data is fundamentally synthetic and operates under a closed-world assumption. It only contains variables and relationships explicitly programmed into the simulation's physics and diplomacy models. Consequently, it lacks the semantic heterogeneity, unpredictability, and massive scale of open-domain knowledge bases like DBpedia or Wikidata. While this ensures flawless validation and reasoning execution, it means the ontology is solving a highly optimized, explicitly bounded domain problem rather than interpreting ambiguous, real-world information.

Knowledge Graph Construction

RDF Model and Triple Structure

The core of the "Ontology Wars" simulation relies on the Resource Description Framework (RDF) to model the state of the world. RDF structures data as a directed, labeled graph composed of nodes and edges, expressed as discrete statements known as triples. Every assertion in the knowledge graph follows a rigid Subject–Predicate–Object structure:

- Subject: The resource being described, represented by a unique URI (e.g., a specific kingdom).
- Predicate: The property or relationship describing the subject (e.g., has a state, possesses a resource).
- Object: The value of the property, which can either be another resource URI (creating an edge to another node) or a Literal value (e.g., an integer or string).

By mapping the transient game state into this semantic web standard, the engine transforms flat JSON variables into a logically queryable network of interconnected concepts.

Representative Examples of RDF Triples

During each simulation turn, the Python engine parses the telemetry_log.json and programmatically generates triples. Below are representative examples serialized in Turtle format (.ttl), demonstrating how data from Turn 1 (e.g., Kingdom_Avalon) is mapped into the graph:

- Class Assignment (Subject - Predicate - Object URI):

  game:Kingdom_Avalon rdf:type game:NPC .

  game:Kingdom_Avalon rdf:type game:Entity .

- Data Property Assignment (Subject - Predicate - Literal):

  game:Kingdom_Avalon game:healthPoints "100"^^xsd:integer .

- Resource Reification and Linking (Complex Object Property):

  Instead of directly assigning gold as a number to the kingdom, the graph creates a specific Gold resource node and links it:

  game:Gold_Kingdom_Avalon rdf:type game:Gold .

  game:Gold_Kingdom_Avalon game:resourceAmount "45"^^xsd:integer .

  game:Kingdom_Avalon game:hasResource game:Gold_Kingdom_Avalon .

- Diplomatic Relations (Turn 2 Example):

  game:Player_Kingdom game:isAlliedWith game:Kingdom_Avalon .

Deployment, Namespace, and Configuration

While traditional semantic applications might rely on external triplestores like GraphDB, this project utilizes a highly dynamic, purely programmatic approach via the RDFlib Python framework to support real-time simulation speeds.

- Namespace Definition: To ensure all URIs are universally unique and resolvable, a custom namespace is bound to the graph at the start of execution:

  GAME = Namespace("http://www.semanticweb.org/cse3226/strategy-game-ontology/v2#")

  g.bind("game", GAME)

- Dataset Loading and TBox Integration: At the beginning of each turn's reasoning phase, an empty in-memory graph (g = Graph()) is instantiated. The static TBox schema is then loaded directly into this graph (g.parse("StrategyGameOntology.owl", format="xml")), ensuring all subsequent ABox assertions inherit the correct logical hierarchy.

- Dynamic Graph Construction: The engine iterates through the active entities in the telemetry log. For each entity, it mints URIs on the fly (e.g., GAME[e["id"]]) and appends the necessary triples (g.add(...)). All numerical values are strictly cast as XSD.integer datatypes to ensure compatibility with SPARQL mathematical filters. Furthermore, the construction pipeline dynamically categorizes the tactical state of each kingdom (assigning game:Safe, game:ExpansionPhase, or game:UnderAttack classes) based on the raw threat level integers before passing the graph to the reasoning engine.

- Serialization and Export: Once the graph is fully constructed, validated via SHACL, and reasoned over via SPARQL, the entire RDF graph is serialized and exported to a local directory (knowledge_graphs/turn_X.ttl). This allows for post-simulation auditing, manual inspection, or future deployment into enterprise triplestores like GraphDB or Apache Jena if persistent hosting is required.

Visual Representation of the Graph Structure

(Note: Please insert a screenshot of your graph here. You can generate a beautiful visualization by opening your StrategyGameOntology.owl or one of the turn.ttl files in Protégé and using the OntoGraf or WebVOWL plugin, or by importing a .ttl file into a free GraphDB local instance and taking a screenshot of the Visual Graph tool).

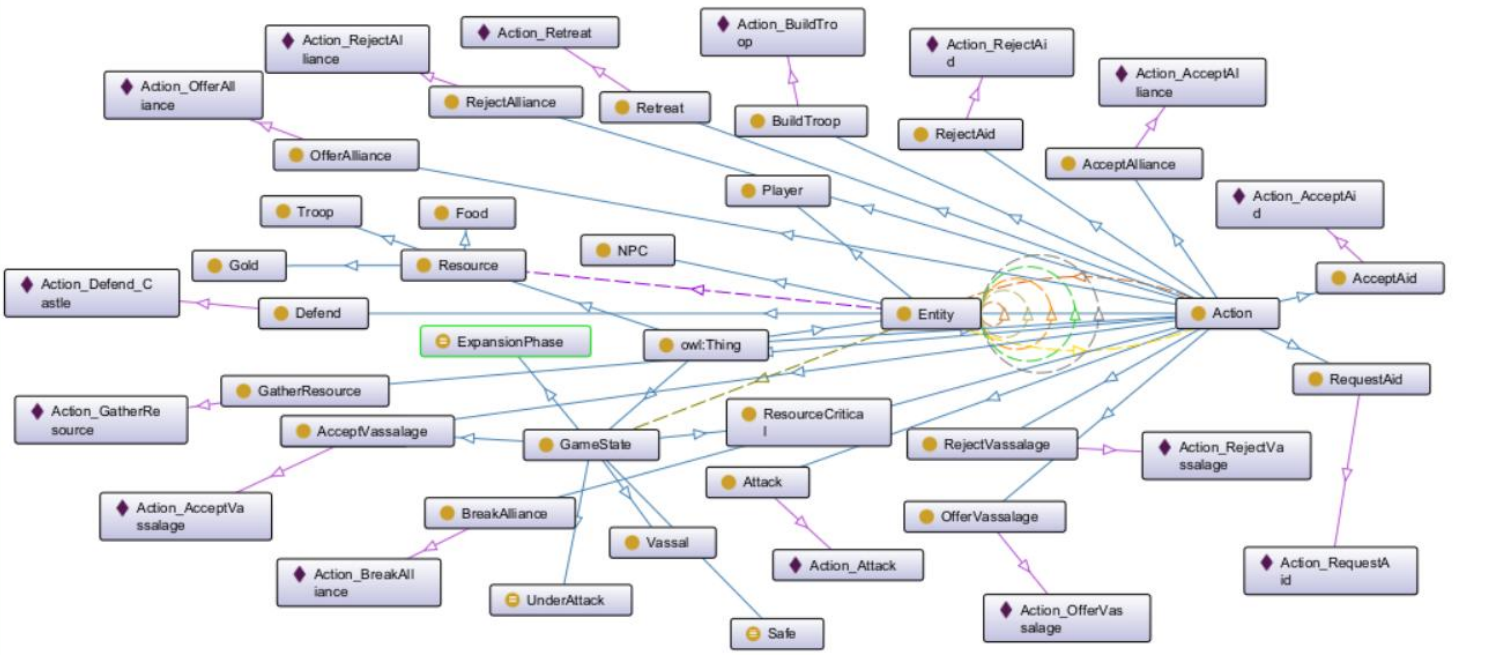

Figure 1: Visual representation of the knowledge graph, illustrating the central Entity node branching out to its GameState, specific Resource nodes, and inter-entity diplomatic relationships.

SPARQL Queries

The querying architecture of "Ontology Wars" is bipartite. It is divided into an active rule-based reasoning engine (INSERT operations) that drives the game logic, and a passive data retrieval layer (SELECT operations) used for graph analysis and aggregation.

Part A: Rule-Based Reasoning via SPARQL (INSERT)

As discussed in the conceptual modeling phase, traditional SWRL rules were replaced with SPARQL UPDATE (specifically INSERT) queries to handle mathematical game physics seamlessly. The Python simulation engine executes 17 distinct INSERT queries at the start of each turn to infer the game:recommendedAction for every entity.

Example Reasoning Query (S6: Expansion Phase Strategy): If a kingdom is in the Expansion Phase and has sufficient food resources (>= 20), the semantic engine infers that it should initiate an attack.

```
PREFIX game: <http://www.semanticweb.org/cse3226/strategy-game-ontology/v2#>
INSERT {
  ?e game:recommendedAction game:Action_Attack
}
WHERE {
  ?e game:hasState ?s .
  ?s a game:ExpansionPhase .
  ?e game:hasResource ?r .
  ?r a game:Food ;
```

```
    game:resourceAmount ?amt .
  FILTER(?amt >= 20)
}
```

This semantic inference mechanism forms the absolute core of our deterministic AI logic, automatically populating the ABox with tactical decisions before the LLM generates the narrative.

Part B: Graph Retrieval and Aggregation (SELECT)

To demonstrate the semantic capabilities of the "Ontology Wars" knowledge graph, 10 SPARQL queries have been implemented. These queries evaluate the ABox against the TBox schema and are categorized into basic retrieval, reasoning, and aggregation operations. All queries relate directly back to the Competency Questions (CQs) defined in Section 2.

The queries below were successfully tested and validated by importing the generated turn_31.ttl files into GraphDB.

Namespace Prefixes used in all queries:

PREFIX game: <http://www.semanticweb.org/cse3226/strategy-game-ontology/v2#>

PREFIX rdf: http://www.w3.org/1999/02/22-rdf-syntax-ns#

Basic Retrieval Queries (Simple Data Extraction)

Query 1: Entity State Classification

Related to: Competency Question 1

Purpose: Retrieves the specific tactical state class (e.g., Safe, UnderAttack) of Kingdom_Avalon based on its current active threat level.

Code:

```
SELECT ?stateClass
WHERE {
  game:Kingdom_Avalon game:hasState ?stateNode .
  ?stateNode rdf:type ?stateClass .
  FILTER(?stateClass != game:GameState) }
```

Expected Output: game:ExpansionPhase (depending on the specific turn's telemetry).

Query 2: Diplomatic Network Retrieval

Related to: Competency Question 3

Purpose: Extracts the explicit identities of all current diplomatic allies bound to the Player_Kingdom.

Code:

```
SELECT ?ally
WHERE {
  game:Player_Kingdom game:isAlliedWith ?ally .
}
```

Expected Output: A list of URIs such as game:Kingdom_ElvenForest.

Query 3: Vassalage Hierarchy Extraction

Related to: Competency Question 3

Purpose: Identifies all current hierarchical protection pacts by retrieving entities that have submitted as vassals and linking them to their overlords.

Code:

```
SELECT ?vassal ?overlord
WHERE {
   ?vassal game:isVassalOf ?overlord .
}
```

Expected Output: ?vassal = game:Kingdom_OrcHorde, ?overlord = game:Kingdom_Avalon.

Query 4: Economic Crisis Detection (Famine vs. Wealth)

Related to: Competency Question 2

Purpose: Identifies entities hoarding gold (>= 40) but suffering from a severe food deficit (<= 20), demonstrating cross-resource relational reasoning.

Code:

```
SELECT ?entity
WHERE {
   ?entity game:hasResource ?goldNode, ?foodNode .

   ?goldNode rdf:type game:Gold ;
         game:resourceAmount ?goldAmt .
   FILTER(?goldAmt >= 40) .

   ?foodNode rdf:type game:Food ;
         game:resourceAmount ?foodAmt .
   FILTER(?foodAmt <= 20) .}
```

Expected Output: game:Kingdom_Avalon (matching Turn 8 logic).

Query 5: Tactical Action Inference (Desperate Retreat)

Related to: Competency Question 4

Purpose: Determines the inferred action for any kingdom classified under the UnderAttack hierarchy that has a critically low troop count (< 20).

Code:

```
SELECT ?entity ?action
WHERE {
   ?entity game:hasState ?stateNode ;
         game:recommendedAction ?action ;
         game:hasResource ?troopNode .
```

```
  ?stateNode rdf:type game:UnderAttack .

  ?troopNode rdf:type game:Troop ;
        game:resourceAmount ?troopAmt .
  FILTER(?troopAmt < 20) .
}
```

Expected Output: ?entity = game:Player_Kingdom, ?action = game:Action_Retreat.

Query 6: Overlord Exploitation Mechanics

Related to: Competency Question 8

Purpose: Identifies subordinate vassal kingdoms that possess enough gold (>= 50) for their structural Overlord to exploit to fund their own army.

Code:

```
SELECT ?overlord ?vassal ?vassalGold
WHERE {
  ?vassal game:isVassalOf ?overlord ;
      game:hasResource ?goldNode .

  ?goldNode rdf:type game:Gold ;
       game:resourceAmount ?vassalGold .
  FILTER(?vassalGold >= 50) .
}
```

Expected Output: ?overlord = game:Kingdom_Avalon, ?vassal = game:Kingdom_ElvenForest, ?vassalGold = 65.

Query 7: Alliance Proposal Viability Assessment

Related to: Competency Question 7

Purpose: Finds entities that have been offered an alliance but checks if they possess the requisite military power (Troops >= 40) to make the pact strategically viable.

Code:

```
SELECT ?offeringEntity ?receivingEntity ?troopAmt
WHERE {
  ?offeringEntity game:offersAllianceTo ?receivingEntity .
  ?receivingEntity game:hasResource ?troopNode .
  ?troopNode rdf:type game:Troop ;
        game:resourceAmount ?troopAmt .
  FILTER(?troopAmt >= 40) .
}
```

Expected Output: ?offeringEntity = game:Player_Kingdom, ?receivingEntity = game:Kingdom_OrcHorde, ?troopAmt = 50.

Query 8: Expansionist Threat Assessment (Average Threat Level)

Related to: Competency Question 6

Purpose: Calculates the global average threat level of all active Non-Player Character (NPC) entities to determine the overall hostility of the simulation environment.

Code:

```
SELECT (AVG(?threat) AS ?averageGlobalThreat)
WHERE {
  ?entity rdf:type game:NPC ;
        game:hasState ?stateNode .
  ?stateNode game:threatLevel ?threat .
}
```

Expected Output: A decimal value representing average threat, e.g., ?averageGlobalThreat = 6.5.

Query 9: Total Military Mobilization per State

Related to: General Domain Analysis

Purpose: Groups entities by their current tactical GameState and calculates the total number of troops mobilized globally within each tactical category using SUM and GROUP BY.

Code:

```
SELECT ?stateClass (SUM(?troopAmt) AS ?totalTroops)
WHERE {
  ?entity game:hasState ?stateNode ;
        game:hasResource ?troopNode .
  ?stateNode rdf:type ?stateClass .
  FILTER(?stateClass IN (game:Safe, game:ExpansionPhase, game:UnderAttack)) .

  ?troopNode rdf:type game:Troop ;
          game:resourceAmount ?troopAmt .
}
GROUP BY ?stateClass
```

Expected Output: Aggregated troop counts per state, e.g., ?stateClass = game:ExpansionPhase, ?totalTroops = 85.

Query 10: Supreme Military Superpower (MAX)

Related to: General Domain Analysis

Purpose: Identifies the single entity with the absolute highest troop count in the current turn by utilizing the MAX aggregation function and ORDER BY limiting.

Code:

```
SELECT ?entity (MAX(?troopAmt) AS ?maxTroops)
WHERE {
   ?entity game:hasResource ?troopNode .
   ?troopNode rdf:type game:Troop ;
            game:resourceAmount ?troopAmt .
}
GROUP BY ?entity
ORDER BY DESC(?maxTroops)
LIMIT 1
```

Expected Output: The top military power, e.g., ?entity = game:Kingdom_OrcHorde, ?maxTroops = 50.

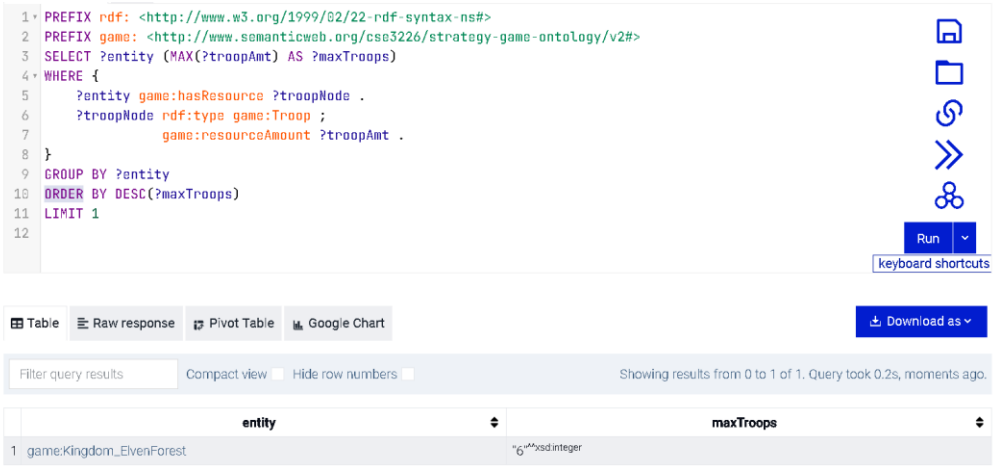

Figure 2: Execution of Aggregation Query in GraphDB

Validation

The Role of SHACL in the Pipeline

In a highly dynamic simulation where the knowledge graph (ABox) is regenerated every turn, ensuring the structural consistency and semantic integrity of the data is paramount. Before the reasoning engine executes any tactical SPARQL rules, the graph must be validated to prevent logical paradoxes (e.g., an entity with negative health or lacking a defined tactical state). To achieve this, we utilized the Shapes Constraint Language (SHACL). Using the pyshacl Python library, the active RDF graph is evaluated against a predefined set of SHACL shapes (the validation schema) at the beginning of every turn.

Defined SHACL Constraints and Shapes

We defined several constraints to enforce datatype correctness, logical boundaries, and cardinality restrictions. Below is the primary shape, game:EntityShape, which targets instances of the game:Entity class:

```
@prefix sh: <http://www.w3.org/ns/shacl#> .
@prefix game: <http://www.semanticweb.org/cse3226/strategy-game-ontology/v2#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
game:EntityShape a sh:NodeShape ;
```

```
sh:targetClass game:Entity ;
# 1. Datatype and Property Constraint (MinInclusive)
sh:property [
   sh:path game:healthPoints ;
   sh:datatype xsd:integer ;
   sh:minInclusive 0 ;
   sh:message "Violation: Health Points cannot drop below 0!" ;
] ;
# 2. Cardinality Restriction (Exact Count)
sh:property [
   sh:path game:hasState ;
   sh:minCount 1 ;
   sh:maxCount 1 ;
   sh:message "Violation: Each kingdom must have exactly 1 GameState!" ;
] .
```

- Property and Datatype Constraint (sh:minInclusive, sh:datatype): The first property shape enforces that the game:healthPoints property must strictly be an integer and cannot fall below zero. In a combat simulation, subtracting damage from health can mathematically result in negative numbers. This constraint ensures that the graph biologically and logically represents "elimination" (0 HP) rather than mathematically impossible negative life.
- Cardinality Restriction (sh:minCount, sh:maxCount): The second property shape enforces that every active entity must possess exactly one game:hasState relationship. It prevents an anomaly where a kingdom is simultaneously Safe and UnderAttack (which would trigger conflicting reasoning rules) or exists in a vacuum with zero states.

Validation Results and Issue Resolution

During the automated execution of the engine (engine_v2.py), the pyshacl validator returns a Boolean conforms flag alongside a detailed results graph.

During the initial development and testing phases, the validation engine successfully caught a critical anomaly: Health Point Violations. When a massive army attacked a weak entity (e.g., the Orc Horde attacking the Player Kingdom), the damage calculation pushed the kingdom's health to -15. pyshacl immediately flagged this, outputting: Validation Report: Conforms: False - Message: "Violation: Health Points cannot drop below 0!"

How it was addressed: Instead of allowing the semantic reasoner to break, this SHACL violation prompted a correction in the upstream Python physics engine. The combat resolution and upkeep logic was updated to use a bounding function (entity["healthPoints"] = max(0, entity["healthPoints"] - damage)). Following this programmatic correction, the SHACL validation consistently passes (Conforms: True), guaranteeing that the SPARQL reasoning engine operates only on a mathematically and logically sound knowledge graph throughout all 30+ turns of the simulation.

System Pipeline and Workflow

Rather than building a standard Natural Language-to-SPARQL query interface, this project implements a Knowledge-Grounded Narrative Generation pipeline. The system bridges formal semantic reasoning with natural language processing to create an interactive, explainable AI storyteller.

The workflow operates as follows:

1. Data Extraction: The Python reasoning engine (engine_v2.py) resolves all SPARQL UPDATE queries and outputs the validated state of the Knowledge Graph (the ABox instances, resources, and recommended actions) into a structured telemetry_log.json file.
2. Context Injection: The Streamlit-based web application (app.py) reads this structured data. When a user selects a specific historical turn, the application extracts the exact sub-graph state for that tick.
3. LLM Invocation: The application utilizes the google.generativeai library to call the Gemini 2.5 Flash model. The extracted semantic data is injected directly into the LLM's context window alongside a carefully engineered system prompt.
4. Output Generation: The LLM parses the structured data and synthesizes it into natural language, displaying the formal ontological facts as an epic fantasy chronicle on the user interface.

Prompt Design

The prompt is dynamically constructed within the application to enforce strict behavioral rules on the LLM. It utilizes persona adoption and thematic constraints to guide the output:

"You are a master fantasy novelist. Transform the following strategy game turn data into an immersive, epic, and dramatic chapter of a novel. The data is generated by an OWL-based Semantic Web reasoning engine. Focus on the political tension, wars, alliances, and resource crises (like famine). Use expressive language, dramatic pacing, and relevant emojis to enhance the reading experience. Keep it engaging but concise (around 2-3 paragraphs). [Language Instruction: English or Turkish]. Turn Data: {JSON_Payload}"

Strategies to Reduce Hallucinations

A major challenge with Large Language Models is their tendency to "hallucinate" or invent non-existent facts. This project drastically reduces hallucinations using three specific strategies:

1. Absolute Data Grounding: The LLM is not asked to invent a story or simulate a game; it is explicitly instructed to transform the provided data. Every action described in the text (e.g., a retreat, a formed alliance, an extortion of a vassal) is strictly grounded in the SPARQL query results that were fed into its prompt.
2. Pre-LLM Structural Validation (SHACL): Because the data is validated by SHACL constraints before being logged, the LLM is never exposed to logical paradoxes (such as an entity having negative health or missing states). The LLM processes a mathematically and logically sound dataset, preventing cascading hallucinations caused by faulty input contexts.
3. Constrained Generative Space: By explicitly stating that the data comes from an "OWL-based Semantic Web reasoning engine," the prompt frames the

LLM's task as a translation from data to prose, acting as a strict semantic boundary.

Example Interaction

Below is a demonstration of how the system translates formal graph data into grounded natural language during Turn 5.

Input Data (Knowledge Graph Extract via SPARQL):

JSON


```
{
  "Kingdom_OrcHorde": {"healthPoints": 70, "gold": 44, "food": 0, "troop": 0, "threatLevel": 6},
  "Kingdom_Avalon": {"healthPoints": 100, "gold": 86, "food": 19, "troop": 16, "threatLevel": 7},
  "sparql_decisions": {
    "Kingdom_Avalon": "Action_Attack",
    "Kingdom_OrcHorde": "Action_Retreat"
  }
}
```


System Output:

The fifth turn of the Great Strife dawned with a grim pallor over the land, marked by desperation and rising aggression. Within the beleaguered walls of the Player Kingdom, the famine bit deep 💀. Granaries stood empty, the last crumbs devoured, yet by a desperate decree, King Theron ordered the last coffers of gold poured into the smithies. "We may starve, but we shall not die defenseless!" he roared, though the hollowness in his own stomach echoed in his voice. With zero troops and zero food, his kingdom teetered on the precipice, clinging to a sliver of hope by forging steel even as its populace withered.

Meanwhile, the mighty Kingdom of Avalon, resplendent in its untouched prosperity, made its move. With a formidable host of sixteen legions, their banners unfurled across the plains, Avalon launched a swift, decisive attack! ⚔️ The air crackled with the unspoken target, a clear declaration of continued war. Elsewhere, the usually fearsome Orc Hordes, reduced to a skeletal shell by their own crippling famine and utter lack of warriors, were forced into a humiliating retreat 😩. Their withdrawal left a perilous power vacuum, a tantalizing prize for any ambitious neighbor. Amidst this maelstrom of hunger and war, the ancient Elven Forest remained a bastion of calm. Its sylvan borders held firm as the Elven lords prudently continued to gather resources 🌿, their ten elite archer companies standing ready, watching the world burn around them with ancient, knowing eyes. The stage was set for a brutal continuation of the war, where empires could crumble into dust.

Evaluation, Discussion and Conclusion

Evaluation and Discussion

From a conceptual and technical perspective, "Ontology Wars: Epic Chronicles" successfully demonstrates that autonomous agent behaviors in complex environments can be driven entirely by formal semantic logic rather than opaque heuristic algorithms. The quality of the ontology is highly robust in terms of consistency and correctness. This is primarily due to the strict enforcement of SHACL constraints prior to any reasoning step, which successfully prevented logical paradoxes such as negative health points or contradictory tactical states. The Knowledge Graph construction and SPARQL reasoning pipeline performed exceptionally well; utilizing SPARQL

UPDATE rules as a pseudo-SWRL reasoning engine proved to be an effective method for deducing complex tactical decisions (e.g., overlord resource exploitation, famine-induced retreats) dynamically at runtime. Furthermore, the integration with the Gemini 2.5 Flash LLM was highly effective. By explicitly grounding the LLM prompts in the validated JSON telemetry extracted from the knowledge graph, we successfully mitigated the persistent issue of LLM hallucination, yielding narratives that were both structurally accurate and contextually rich.

However, the system does have inherent limitations. A major modeling constraint is its reliance on a "closed-world" assumption; because the data is synthetically generated by the Python engine, the ontology is highly specialized and lacks the messy, heterogeneous completeness of open-world datasets. Scalability presents another potential technical limitation. While the in-memory rdflib graph construction and SPARQL reasoning execute seamlessly for a localized, thirty-turn simulation with four entities, scaling the engine to handle thousands of concurrent autonomous agents would likely introduce significant computational bottlenecks and memory overhead, exposing the limitations of in-memory triplestores compared to enterprise-grade graph databases.

Conclusion

In conclusion, this project successfully bridged the gap between deterministic Semantic Web technologies and generative artificial intelligence, delivering a transparent, rule-driven simulation engine where every AI decision is logically traceable and formally verifiable. Throughout the development of this system, we gained profound, hands-on experience in the complete knowledge engineering lifecycle: from theoretical TBox modeling in Protégé to dynamic ABox instantiation in Python, and mastering advanced SPARQL queries for automated reasoning. Furthermore, we learned how structured knowledge graphs serve as the ultimate grounding mechanism to control, direct, and explain LLM outputs. For future extensions, the system could be significantly improved by migrating the in-memory graph to a persistent, scalable backend like GraphDB, expanding the ontology to include a granular economic trade network, and developing a bi-directional interface where a human player's natural language commands are translated back into SPARQL INSERT statements to alter the graph in real-time.

References and format

Allemang, D., Hendler, J., & Gandon, F. (2020). Semantic Web for the Working Ontologist: Effective Modeling in RDFS and OWL (3rd ed.). ACM Books.

Biessmann, F. (2024). PySHACL: A Python SHACL validator (Version 0.25.0) [Computer software]. GitHub. https://github.com/RDFLib/pyshacl

Google. (2025). Gemini 2.5 Flash API documentation. Google DeepMind. https://ai.google.dev/gemini-api/docs

Horrocks, I., Patel-Schneider, P. F., & van Harmelen, F. (2003). From SHIQ and RDF to OWL: The making of a Web Ontology Language. Web Semantics: Science, Services and Agents on the World Wide Web, 1(1), 7-26.

Knublauch, H., & Kontokostas, D. (Eds.). (2017). Shapes Constraint Language (SHACL). W3C Recommendation. https://www.w3.org/TR/shacl/

McGuinness, D. L., & Van Harmelen, F. (2004). OWL Web Ontology Language overview. W3C Recommendation, 10(10), 2004. https://www.w3.org/TR/owl-features/

Prud'hommeaux, E., & Seaborne, A. (Eds.). (2008). SPARQL Query Language for RDF. W3C Recommendation. https://www.w3.org/TR/rdf-sparql-query/

RDFLib Team. (2025). RDFLib: A Python library for working with RDF (Version 7.0.0) [Computer software]. https://rdflib.readthedocs.io/

Streamlit Development Team. (2024). Streamlit framework for interactive web applications [Computer software]. https://docs.streamlit.io/

Appendix

Appendix A: Application User Interface (Streamlit Dashboard)

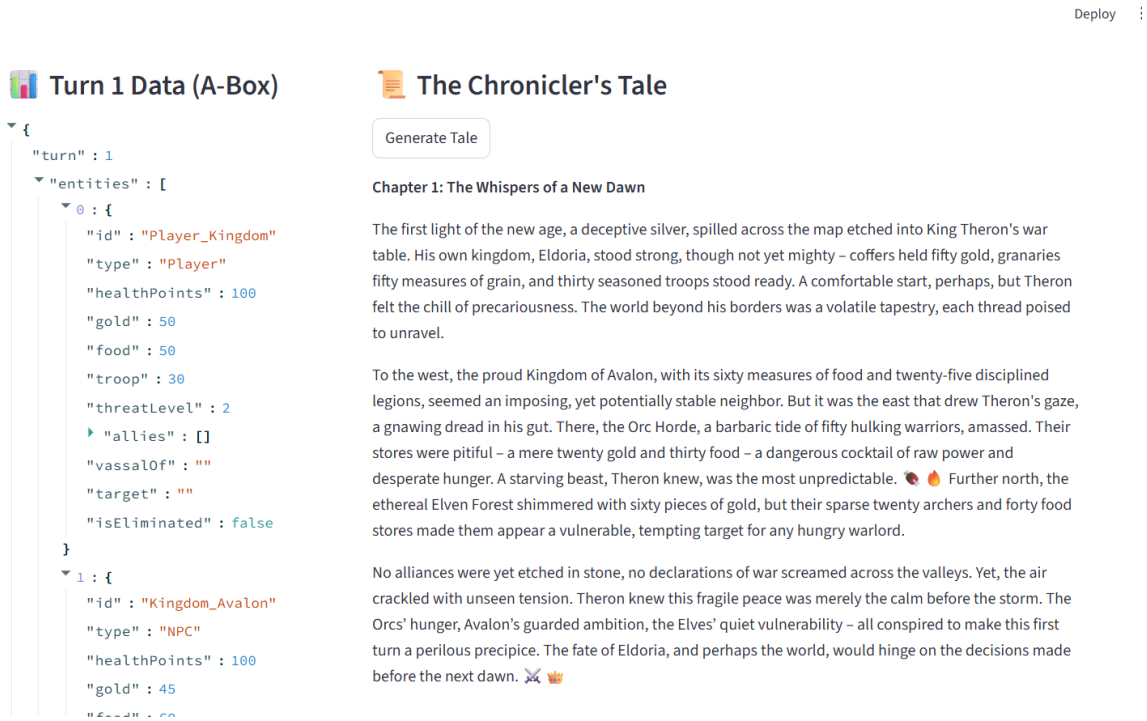

Figure 2 "Ontology Wars" web interface. The sidebar manages configuration, while the main dashboard displays the turn's JSON Knowledge Graph (ABox) and the LLM-synthesized chronicle.

Appendix B: Complete SHACL Validation Schema

While specific constraints were discussed in Section 7 (Validation), the complete SHACL graph utilized by the pyshacl validator within the Python engine (engine_v2.py) is provided below. This schema is parsed dynamically into a Turtle graph before the semantic reasoning phase to ensure that all instantiated entities maintain non-negative health points and possess exactly one tactical state.

```
@prefix sh: <http://www.w3.org/ns/shacl#> .
@prefix game: <http://www.semanticweb.org/cse3226/strategy-game-ontology/v2#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
game:EntityShape a sh:NodeShape ;
    sh:targetClass game:Entity ;
    sh:property [
        sh:path game:healthPoints ;
        sh:datatype xsd:integer ;
        sh:minInclusive 0 ;
```

```
    sh:message "Health Points cannot drop below 0!" ;
  ] ;
  sh:property [
    sh:path game:hasState ;
    sh:minCount 1 ;
    sh:maxCount 1 ;
    sh:message "Each kingdom must have exactly 1 GameState!" ;
  ] .
```

Appendix C: LLM System Prompt Configuration

To ensure the Gemini 2.5 Flash model adhered strictly to the structural constraints of the knowledge graph and avoided narrative hallucinations, the following dynamic prompt was engineered within the app.py application logic. The {lang_instruction} and {json.dumps(turn_info)} variables are injected dynamically at runtime, effectively grounding the LLM's generative capabilities in formal, validated semantic data.

```
prompt = f"""
You are a master fantasy novelist. Transform the following strategy game turn data into an immersive, epic, and dramatic chapter of a novel.
The data is generated by an OWL-based Semantic Web reasoning engine.
Focus on the political tension, wars, alliances, and resource crises (like famine).
Use expressive language, dramatic pacing, and relevant emojis to enhance the reading experience.
Keep it engaging but concise (around 2-3 paragraphs).
{lang_instruction}
Turn Data: {json.dumps(turn_info)}
"""
```

WIDOCO Link: Ontology Documentation generated by WIDOCO

GitHub Link: keremmozakca/StrategyGame-Ontology-Framework